```
# Passive AI Assistant — Edge Manifest V3 Extension

A lightweight Microsoft Edge extension that provides **Level 0 Passive
Intelligence** for any webpage. Summarize pages, explain selected text, and
extract structured key information — all from the browser toolbar.

---

## Table of Contents

- [Overview](#overview)
- [Features](#features)
- [Architecture](#architecture)
- [File Structure](#file-structure)
- [Installation](#installation)
- [Usage](#usage)
  - [Summarize Page](#summarize-page)
  - [Explain Selection](#explain-selection)
  - [Extract Key Info](#extract-key-info)
  - [Copy and Clear](#copy-and-clear)
  - [Settings](#settings)
- [Mock Mode vs Live API Mode](#mock-mode-vs-live-api-mode)
  - [Mock Mode (Default)](#mock-mode-default)
  - [Live API Mode](#live-api-mode)
  - [API Request Format](#api-request-format)
  - [API Response Schema](#api-response-schema)
- [Content Extraction Strategy](#content-extraction-strategy)
- [Permissions](#permissions)
- [Safety and Privacy](#safety-and-privacy)
- [Error Handling](#error-handling)
- [Browser Compatibility](#browser-compatibility)
- [Development Guide](#development-guide)
  - [Project Setup](#project-setup)
  - [Making Changes](#making-changes)
  - [Build Order Reference](#build-order-reference)
- [Acceptance Criteria](#acceptance-criteria)
- [Prompt and Response Contract for LLM Integration](#prompt-and-response-
contract-for-llm-integration)
  - [Summarize Contract](#summarize-contract)
  - [Explain Contract](#explain-contract)
  - [Extract Contract](#extract-contract)
- [Known Limitations](#known-limitations)
- [Roadmap](#roadmap)
- [License](#license)

---

## Overview

Passive AI Assistant is a **Manifest V3** Edge extension built with **native
JavaScript, HTML, and CSS** — no frameworks, no build tools, no external
```

```
dependencies. It operates in a strictly passive, read-only mode: it reads the
current page content when you ask it to, processes that content, and displays
structured results in the popup. It never modifies pages, submits forms,
navigates autonomously, or performs any action without explicit user
initiation.

The extension ships with **mock mode enabled by default**, providing
deterministic heuristic-based responses so you can test the full end-to-end
flow without any external API. When you are ready, you can connect it to a real
AI backend through the settings page.

---

## Features

- **Summarize Page** — Extracts the main content from the current tab and
returns a concise summary with bullet-point key takeaways.
- **Explain Selection** — Takes your highlighted text and provides a plain-
English explanation or paraphrase.
- **Extract Key Info** — Identifies the page type, named entities (people,
organizations, dates, numbers), key claims, risks, and actions mentioned.
- **Mock Mode** — Fully functional offline testing with built-in heuristic
responses. No API key required.
- **Live API Mode** — Optional integration with any AI backend that accepts the
documented request/response contract.
- **Smart Content Extraction** — Prioritizes semantic HTML elements
(`<article>`, `<main>`, common content containers) and strips navigation,
footers, ads, cookie banners, and other noise.
- **Context Menu Integration** — Right-click on selected text to get a badge
hint for the Explain Selection action.
- **Copy and Clear** — One-click copy of results to clipboard and clear to
reset the panel.
- **Settings Page** — Toggle mock/live mode, configure API endpoint and API
key, all persisted in local storage.

---

## Architecture

The extension follows a clean three-layer message-passing architecture aligned
with Manifest V3 requirements.
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│ POPUP (UI) │
│ popup.html · popup.css · popup.js │
│ User clicks action → sends message to background │
│ Receives structured result → renders in result panel │
└──────────────────────────┬──────────────────────────────────────────────
│ chrome.runtime.sendMessage
```

```
▼
┌──────────────────────────────────────────────────────────────────────────────┐
│ BACKGROUND (Service Worker) │
│ background.js │
│ Receives action request from popup │
│ Orchestrates content extraction from content script │
│ Reads settings from chrome.storage.local │
│ Routes to Mock AI layer or Live API layer │
│ Returns structured response to popup │
└──────────────────┬──────────────────────────────────────┬──────────────────────────────────┘
│ chrome.tabs.sendMessage │ fetch (live mode only)
▼ ▼
┌──────────────────────────────────┐ ┌──────────────────────────────────┐
│ CONTENT SCRIPT │ │ EXTERNAL AI API │
│ content.js │ │ (optional, user- │
│ Runs in page context │ │ configured endpoint) │
│ Extracts DOM text │ └──────────────────────────────────┘
│ Returns title, URL, │
│ selection, content │
└──────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────────────┐
│ OPTIONS PAGE │
│ options.html · options.css · options.js │
│ Toggle mock mode · Set API endpoint · Set API key │
│ Persists to chrome.storage.local │
└──────────────────────────────────────────────────────────────────────────────┘
```

```
**Data flow for every action:**

1. User clicks a button in the popup.
2. Popup sends an `extractContent` message to the background service worker.
3. Background sends an `extractPageContent` message to the content script
running in the active tab.
4. Content script extracts text from the DOM, captures any selected text, and
responds.
5. Background receives the extracted content, reads settings, and routes to
either the Mock AI layer or the Live API.
6. Background sends the structured result back to the popup.
7. Popup renders the result in the result panel.

---

## File Structure
```

```
edge-passive-ai-mvp/
├── manifest.json Extension manifest (MV3)
├── popup.html Popup UI markup
├── popup.css Popup styles
├── popup.js Popup logic and rendering
├── background.js Service worker: routing, mock AI, live API
├── content.js Content script: DOM text extraction
├── options.html Settings page markup
├── options.css Settings page styles
├── options.js Settings page logic
├── README.md This file
└── icons/
├── icon16.png 16×16 toolbar icon
├── icon48.png 48×48 management page icon
└── icon128.png 128×128 install/store icon
```

```
---

## Installation

### Prerequisites

- Microsoft Edge (Chromium-based, version 88 or later)
- Three PNG icon files (16×16, 48×48, 128×128 pixels)

### Steps

1. **Clone or download** this repository to a local folder.

2. **Add icon files.** Place your PNG icons in the `icons/` directory:
   - `icons/icon16.png` (16×16 px)
   - `icons/icon48.png` (48×48 px)
   - `icons/icon128.png` (128×128 px)

   If you do not have icons, you can generate a simple set using [favicon.io]
(https://favicon.io/) or create solid-color squares with a letter "P" in any
image editor.

3. **Open Edge** and navigate to `edge://extensions/`.

4. **Enable Developer mode** using the toggle (typically found in the bottom-
left or sidebar).

5. Click **Load unpacked** and select the `edge-passive-ai-mvp` folder.

6. The **Passive AI** icon should appear in your Edge toolbar. If it does not
appear immediately, click the puzzle piece icon in the toolbar and pin the
extension.
```

```
7. **Navigate to any normal webpage** (not `edge://`, `chrome://`, or `about:`
pages) and click the extension icon to open the popup.

---

## Usage

### Summarize Page

1. Navigate to any webpage with readable content.
2. Click the extension icon to open the popup.
3. Click the **Summarize Page** button.
4. The extension extracts the main visible text from the page.
5. A summary and key points appear in the result panel.

### Explain Selection

1. On any webpage, **select (highlight) some text** with your mouse.
2. Click the extension icon to open the popup.
3. Click the **Explain Selection** button.
4. The extension retrieves your selected text and returns a plain-English
explanation.
5. If no text is selected, an informative error message appears.

### Extract Key Info

1. Navigate to any webpage.
2. Click the extension icon to open the popup.
3. Click the **Extract Key Info** button.
4. The extension returns structured information:
   - **Page Type** — a classification of the page (News, Blog, Wiki, E-
commerce, General).
   - **Key Points** — the first few meaningful sentences.
   - **Entities** — detected names, organizations, dates, numbers.
   - **Signals** — claims, risks, and actions mentioned on the page.

### Copy and Clear

- **📋 Copy** — Copies the current result text to your clipboard.
- **🗑 Clear** — Clears the result panel and resets the status bar.

### Settings

- Click the **⚙ Settings** button in the popup to open the options page.
- See the [Mock Mode vs Live API Mode](#mock-mode-vs-live-api-mode) section for
configuration details.

---

## Mock Mode vs Live API Mode

### Mock Mode (Default)
```

```
Mock mode is **enabled by default**. In this mode, the extension uses built-in
JavaScript heuristics to generate deterministic responses. No network requests
are made. This allows you to:

- Test the full extension flow end-to-end.
- Verify UI rendering, error handling, and state transitions.
- Develop and debug without needing an API key or backend.

**How mock responses work:**

| Action | Mock Logic |
|--------|-----------|
| Summarize Page | Returns the first 1–2 sentences as a summary and up to 5
sentences as key points. |
| Explain Selection | Wraps the selected text in a deterministic paraphrase
template. Multi-sentence selections are broken into numbered points. |
| Extract Key Info | Detects capitalized multi-word phrases as possible
entities, matches date and number patterns with regex, and flags sentences
containing claim/risk/action keywords. |

### Live API Mode

To use a real AI backend:

1. Open the **Settings** page (⚙ button in the popup).
2. **Uncheck** the "Mock Mode" toggle.
3. Enter your **API Endpoint** URL (e.g., `https://your-server.com/analyze`).
4. Enter your **API Key** (sent as a Bearer token in the `Authorization`
header).
5. Click **Save Settings**.

The extension validates that an API endpoint is provided when mock mode is off.

### API Request Format

All actions send a `POST` request with `Content-Type: application/json`:

```json
{
  "action": "summarizePage | explainSelection | extractKeyInfo",
  "title": "Page title",
  "url": "https://example.com/page",
  "content": "Extracted visible page text...",
  "selection": "User-selected text (only for explainSelection)"
}
```
```

API Response Schema

The extension expects the following JSON response structure:

```
{
  "pageType": "string",
  "summaryShort": "string",
  "keyPoints": ["string"],
  "entities": {
    "people": ["string"],
    "organizations": ["string"],
    "products": ["string"],
    "dates": ["string"],
    "locations": ["string"],
    "numbers": ["string"]
  },
  "signals": {
    "mainClaims": ["string"],
    "risks": ["string"],
    "actionsMentioned": ["string"]
  },
  "explanation": "string",
  "confidenceNote": "Derived only from supplied page content."
}
```

Not all fields are required for every action. The popup renderer gracefully handles missing fields.

Content Extraction Strategy

The content script (`content.js`) extracts text from the current page using a prioritized strategy:

Priority	Target	Description
1	`<article>`	Semantic article element — most reliable for news and blog content.
2	`<main>`	Semantic main content area.
3	Common containers	`#content`, `#main-content`, `.content`, `.main-content`, `.post-content`, `.entry-content`, `.article-body`, `[role='main']`
4	`document.body`	Full body text as final fallback.

Noise removal: Before extracting text, the script clones the target subtree and removes the following elements:

- `nav`, `footer`, `aside`, `.sidebar`
- `[role='navigation']`, `[role='banner']`, `[role='contentinfo']`
- `.cookie-banner`, `.cookie-consent`, `#cookie-banner`, `#cookie-consent`
- `.ad`, `.ads`, `.advertisement`
- `script`, `style`, `noscript`, `svg`, `iframe`, `video`, `audio`, `canvas`

Post-processing:

- Excessive whitespace is collapsed (3+ newlines reduced to 2, multiple spaces reduced to 1).

- Content is truncated to **15,000 characters** to keep payloads manageable.
- Uses `textContent` instead of `innerText` on cloned nodes for reliability on detached DOM elements.

Permissions

Permission	Reason
`activeTab`	Access the currently active tab when the user clicks the extension icon.
`scripting`	Programmatically inject the content script if it was not loaded by the manifest match pattern.
`storage`	Persist user settings (mock mode toggle, API endpoint, API key) in `chrome.storage.local`.
`contextMenus`	Add a right-click menu item for "Explain Selection" convenience.

No host permissions are declared in the manifest. If your live API endpoint requires it, you may need to add the API domain to `host_permissions` in `manifest.json`.

Safety and Privacy

This extension is designed with a **strictly passive, read-only** model:

- **No autonomous actions.** The extension only acts when the user explicitly clicks a button.
- **No page modification.** The DOM is never altered, no forms are submitted, no navigation occurs.
- **No persistent memory.** No data is stored beyond the current request. Settings (mock mode, API config) are the only persisted values.
- **No cross-tab orchestration.** Only the active tab is accessed, only when requested.
- **Content grounding.** All outputs (mock or live) are framed as derived from the supplied/extracted page content only. The mock layer does not fabricate information beyond what is present in the extracted text.
- **No tracking or analytics.** The extension does not collect, transmit, or store any user data, browsing history, or page content beyond the immediate processing flow.
- **API key handling.** If provided, the API key is stored in `chrome.storage.local` (browser-local, not synced) and sent only to the user-configured endpoint.

Error Handling

The extension handles the following error cases with human-readable messages:

Scenario	Behavior
No active tab found	Status bar shows error message.

Scenario	Behavior
Restricted page (`edge://`, `chrome://`, `about:`)	"Cannot analyze restricted browser pages."
Content script injection fails	"Cannot access this page." with the specific error.
Empty content extraction	"Could not extract content from this page."
No text selected (Explain Selection)	"No text is selected on the page. Please select some text first."
Missing API endpoint (live mode)	"API endpoint not configured. Please check Settings."
API returns non-200 status	"API returned status {code}: {statusText}"
Network failure	The native fetch error message is displayed.
Unknown action type	"Unknown action: {action}"
Invalid message payload	"Invalid action request: missing action or payload."

All errors appear in the popup status bar with a red error state.

Browser Compatibility

Browser	Supported	Notes
Microsoft Edge (Chromium)	Yes	Primary target. Version 88+ required for MV3.
Google Chrome	Yes	Fully compatible — same Chromium MV3 APIs.
Brave	Yes	Chromium-based, should work without modification.
Firefox	No	Firefox uses a different MV3 implementation.
Safari	No	Different extension model.

Development Guide

Project Setup

No build tools are required. The extension uses only native web technologies.

1. Clone the repository.
2. Add icon files to `icons/`.
3. Load as an unpacked extension in Edge (see Installation).
4. Make changes to any file and reload the extension from `edge://extensions/` to see updates.

Making Changes

- **Popup UI changes:** Edit `popup.html`, `popup.css`, or `popup.js`. Close and reopen the popup to see changes.
- **Content script changes:** Edit `content.js`. Reload the extension from `edge://extensions/` and refresh the target page.
- **Background worker changes:** Edit `background.js`. Reload the extension from `edge://extensions/`.
- **Settings page changes:** Edit `options.html`, `options.css`, or `options.js`. Close and reopen the options page.
- **Manifest changes:** Edit `manifest.json`. Reload the extension from `edge://extensions/`.

Build Order Reference

If rebuilding from scratch, follow this sequence:

Phase	Steps
Phase 1: Shell	Create `manifest.json`, popup UI files, background worker, content script.
Phase 2: Extraction	Implement active tab query, content script injection, text extraction and return.
Phase 3: Mock Intelligence	Add mock mode default, implement mock summarize/explain/extract logic.
Phase 4: UX Polish	Add loading/error/success states, structured result rendering, copy and clear buttons.
Phase 5: Settings and API	Build options page, add live API request path in background worker.

Acceptance Criteria

Functional

- ☑ Extension loads unpacked in Edge without errors.
- ☑ Popup opens successfully on normal webpages.
- ☑ **Summarize Page** returns a result using extracted current page text.
- ☑ **Explain Selection** returns an informative message if no text is selected, otherwise displays an explanation.
- ☑ **Extract Key Info** returns a structured result with page type, entities, and signals.
- ☑ Settings page saves and reloads configuration from storage.
- ☑ Mock mode works without any external services.

UX

- ☑ Popup shows clear status transitions (idle → loading → success/error).
- ☑ Results are readable and scrollable.
- ☑ Copy button copies visible result text to clipboard.
- ☑ Errors are human-readable.

Safety

- ☑ No automatic browsing actions occur.
- ☑ No page submission or modification occurs.
- ☑ Output is framed as derived from supplied/extracted page content only.

Prompt and Response Contract for LLM Integration

When connecting a real AI backend, the model layer should follow these contracts.

System behavior requirements:

- Analyze only the supplied webpage text.
- Do not infer unstated facts.
- Return structured JSON only.
- Use empty arrays or `"not stated"` when data is absent.

Summarize Contract

Input:

```
{
  "action": "summarizePage",
  "title": "Page title",
  "url": "https://example.com",
  "content": "Extracted page text..."
}
```

Output:

```
{
  "summaryShort": "A concise 1-2 sentence summary.",
  "keyPoints": [
    "First key takeaway.",
    "Second key takeaway."
  ],
  "confidenceNote": "Derived only from supplied page content."
}
```

Explain Contract

Input:

```
{
  "action": "explainSelection",
```

```
  "title": "Page title",
  "url": "https://example.com",
  "selection": "The user-selected text to explain.",
  "content": "Optional surrounding page context..."
}
```

Output:

```
{
  "explanation": "A plain-English explanation of the selected text.",
  "confidenceNote": "Derived only from supplied page content."
}
```

Extract Contract

Input:

```
{
  "action": "extractKeyInfo",
  "title": "Page title",
  "url": "https://example.com",
  "content": "Extracted page text..."
}
```

Output:

```
{
  "pageType": "News/Article",
  "keyPoints": [
    "First key point.",
    "Second key point."
  ],
  "entities": {
    "people": ["Name One", "Name Two"],
    "organizations": ["Org One"],
    "products": [],
    "dates": ["January 15, 2026"],
    "locations": ["New York"],
    "numbers": ["$4.5 billion", "23%"]
  },
  "signals": {
    "mainClaims": ["The company claims market leadership."],
    "risks": ["Regulatory risk was cited."],
    "actionsMentioned": ["Plans to expand into Europe."]
  },
```

```
  "confidenceNote": "Derived only from supplied page content."
}
```

Known Limitations

- **Mock mode is heuristic-only.** Mock responses use simple string manipulation and regex patterns. They are not intelligent and will not produce accurate summaries or entity extraction on complex pages. They exist purely for testing the extension flow.
- **Content extraction is best-effort.** Heavily JavaScript-rendered pages (SPAs) may yield limited or empty content because the content script reads the DOM at the time of extraction. If critical content is loaded asynchronously after page idle, it may be missed.
- **15,000 character content limit.** Very long pages are truncated. This is intentional for MVP payload management but may omit important content at the end of long articles.
- **Entity detection in mock mode is naive.** The capitalized-phrase regex will surface many false positives (e.g., sentence-starting words). The people, products, and locations arrays are always empty in mock mode because distinguishing these categories requires real NLP.
- **No persistent memory.** Each action is independent. The extension does not remember previous requests or build context across actions.
- **Restricted pages.** The extension cannot operate on browser internal pages (`edge://`, `chrome://`, `about:`) or other extension pages.
- **Context menu limitation.** MV3 does not allow programmatic popup opening. The context menu "Explain Selection" item sets a badge hint but the user must still click the extension icon to open the popup.

Roadmap

Future enhancements (not part of the current MVP):

- **Level 1 Active Intelligence** — Allow the extension to offer contextual suggestions based on page content.
- **Multi-model support** — Configure different AI backends for different actions.
- **Response caching** — Cache results per URL to avoid redundant extraction and API calls.
- **Export results** — Save results as Markdown, JSON, or plain text files.
- **Side panel mode** — Render results in an Edge side panel instead of the popup for persistent viewing.
- **Keyboard shortcuts** — Bind actions to configurable keyboard shortcuts.
- **Enhanced entity extraction** — Improve mock mode with more sophisticated NLP heuristics.
- **Batch mode** — Process multiple open tabs sequentially.

License

This project is provided as-is for educational and internal development purposes. No license is currently specified. Add your preferred license file as needed.